ARTIFICIAL INTELLIGENCE PRACTICAL REPORT | TE COMPUTER ENGINEERING

Implementation of A* Search Algorithm for 8-Puzzle Problem

1. AIM

To design and implement the **A* search algorithm** in Python to find the optimal solution for solving an 8-Puzzle game using heuristic evaluation functions.

2. PRACTICAL PREREQUISITES & REQUIREMENTS

Component	Specification
Programming Language	Python 3.x
Operating System	Any OS (Windows, Linux, macOS)
IDE / Editor	VS Code, Jupyter Notebook, or PyCharm
Hardware Requirements	Any modern CPU with at least 4 GB RAM

3. THEORY

3.1 The 8-Puzzle Problem

The 8-puzzle is a classic sliding tile puzzle consisting of a 3×3 grid containing 8 numbered tiles (1 through 8) and one empty space (represented as 0). The goal is to reach a specified target state from an initial state by sliding tiles into the empty space sequentially.

3.2 A* Search Algorithm

A* is an informed search algorithm that finds the shortest path from a starting node to a goal node. It uses a evaluation cost function $f(n)$ to determine the priority of node expansion:

$$f(n) = g(n) + h(n)$$

- $g(n)$: The exact cost to reach node n from the start node (depth of the node in the state space tree).
- $h(n)$: The estimated cost to reach the goal from node n (heuristic function).

3.3 Heuristic Choice: Manhattan Distance

For the 8-puzzle, **Manhattan Distance** is an admissible heuristic (it never overestimates the actual cost). It measures the sum of absolute horizontal and vertical distances of each tile from its target position:

$$h(n) = \sum |x_{current} - x_{goal}| + |y_{current} - y_{goal}|$$

4. PROCEDURE

1. **State Representation:** Represent the 3×3 grid as a tuple of length 9 or a 2D array.

2. **Data Structures:**
   - **Open List (Priority Queue):** Holds nodes to be evaluated, ordered by the lowest $f(n)$ value.
   - **Closed List (Set):** Stores visited states to avoid evaluation loops.
3. **Algorithm Steps:**
   - Push the initial state with $g = 0$ and calculated $h(start)$ into the Open List.
   - Pop the node with the minimum $f(n)$ value from the Open List.
   - If the current state matches the goal state, backtrack using parent pointers to reconstruct the optimal path.
   - Otherwise, add the current state to the Closed List.
   - Generate valid successor states by sliding the empty tile (`UP`, `DOWN`, `LEFT`, `RIGHT`).
   - For each successor, calculate $g(successor) = g(current) + 1$, compute $h(successor)$, and push to the Open List if not already in the Closed List.

5. PYTHON IMPLEMENTATION CODE

```
import heapq

class PuzzleNode:
    def __init__(self, state, parent=None, move="", g=0, h=0):
        self.state = state          # Tuple of 9 elements
        self.parent = parent        # Pointer to parent node
        self.move = move            # Move made to reach this state
        self.g = g                  # Path cost from start node
        self.h = h                  # Heuristic cost to goal
        self.f = g + h              # Total estimated cost

    def __lt__(self, other):
        return self.f < other.f

def get_manhattan_distance(state, goal):
    distance = 0
    for i, val in enumerate(state):
        if val != 0:
            target_idx = goal.index(val)
            r1, c1 = divmod(i, 3)
            r2, c2 = divmod(target_idx, 3)
            distance += abs(r1 - r2) + abs(c1 - c2)
    return distance

def get_neighbors(node, goal):
    neighbors = []
    zero_idx = node.state.index(0)
    r, c = divmod(zero_idx, 3)

    # Valid moves: (row_offset, col_offset, action_name)
    moves = [(-1, 0, "UP"), (1, 0, "DOWN"), (0, -1, "LEFT"), (0, 1, "RIGHT")]

    for dr, dc, move_name in moves:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_zero_idx = nr * 3 + nc
            new_state = list(node.state)
            # Swap empty space with target tile
            new_state[zero_idx], new_state[new_zero_idx] = new_state[new_zero_idx],
new_state[zero_idx]

            new_state_tuple = tuple(new_state)
```

```
            h = get_manhattan_distance(new_state_tuple, goal)
            neighbors.append(PuzzleNode(new_state_tuple, node, move_name, node.g + 1, h))

    return neighbors

def solve_8_puzzle(start_state, goal_state):
    open_list = []
    closed_set = set()

    start_h = get_manhattan_distance(start_state, goal_state)
    start_node = PuzzleNode(start_state, None, "", 0, start_h)

    heapq.heappush(open_list, start_node)

    while open_list:
        current = heapq.heappop(open_list)

        if current.state == goal_state:
            path = []
            while current:
                path.append(current)
                current = current.parent
            return path[::-1]  # Return reversed path (start to goal)

        closed_set.add(current.state)

        for neighbor in get_neighbors(current, goal_state):
            if neighbor.state in closed_set:
                continue
            heapq.heappush(open_list, neighbor)

    return None

def print_grid(state):
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print("-" * 11)

# Execution
if __name__ == "__main__":
    initial = (1, 2, 3, 0, 4, 6, 7, 5, 8)
    goal    = (1, 2, 3, 4, 5, 6, 7, 8, 0)

    print("Initial State:")
    print_grid(initial)

    path = solve_8_puzzle(initial, goal)

    if path:
        print(f"
Solution found in {len(path) - 1} steps:
")
        for step_idx, node in enumerate(path):
            if step_idx > 0:
                print(f"Step {step_idx}: Move {node.move}")
            print_grid(node.state)
    else:
        print("No solution found.")
```

6. PROGRAM OUTPUT

```
Initial State:
 1 2 3
 0 4 6
 7 5 8
-----------

Solution found in 3 steps:

Step 1: Move RIGHT
 1 2 3
 4 0 6
 7 5 8
-----------
Step 2: Move DOWN
 1 2 3
 4 5 6
 7 0 8
-----------
Step 3: Move RIGHT
 1 2 3
 4 5 6
 7 8 0
-----------
```

7. ANALYSIS & PERFORMANCE EVALUATION

- **Time Complexity:** Worst-case $O(b^d)$, where b is the branching factor (≈ 2 to 4) and d is the depth of the optimal solution. Using Manhattan Distance dramatically prunes the state space and reduces expanded nodes compared to uninformed BFS/DFS.
- **Space Complexity:** $O(b^d)$ because the Open and Closed lists store all generated nodes in memory.
- **Optimality & Admissibility:** Since Manhattan Distance is an **admissible heuristic** ($h(n) \leq h^*(n)$) and **consistent** ($h(n) \leq c(n, a, n') + h(n')$), A* search guarantees finding the optimal (shortest path) solution.

8. CONCLUSION

The A* search algorithm was successfully implemented in Python to solve the 8-puzzle problem. The implementation demonstrates how incorporating domain knowledge via heuristic evaluation functions (Manhattan distance) significantly accelerates search space exploration while guaranteeing solution optimality.

9. VIVA QUESTIONS & ANSWERS

Q1: What is the difference between g(n) and h(n) in A* search?

Answer: $g(n)$ is the exact path cost incurred from the initial state to the current node n. $h(n)$ is the estimated heuristic cost from the current node n to the target goal state.

Q2: What makes a heuristic "admissible"? Why is it important?

Answer: A heuristic is admissible if it never overestimates the actual cost required to reach the goal state. Admissibility is crucial because it mathematically guarantees that A* will always find the optimal (shortest path) solution.

Q3: Can Manhattan distance overestimate the true path cost in an 8-puzzle?

Answer: No. Tiles can move at best one grid cell per move. Manhattan distance computes the absolute minimum moves required assuming zero blocking tiles, ensuring $h(n) \leq h^*(n)$ always holds.

Q4: What happens if h(n) = 0 for all nodes?

Answer: The algorithm degrades into Dijkstra's Algorithm (or Uniform Cost Search), expanding nodes based purely on path depth $g(n)$ without heuristic guidance.

Q5: Are all random initial configurations of the 8-puzzle solvable?

Answer: No. Exactly 50% of randomly generated initial configurations are unsolvable. Solvability depends on the inversion count of the tile sequence—an even number of inversions indicates a solvable puzzle, while an odd number means it is mathematically impossible to reach the standard goal state.